COS30002 Artificial Intelligence for Games

Semester 1, 2026
Portfolio - Learning Summary Report

Pattarapol Tantechasa (103883220)

Declaration

I declare that this portfolio is my individual work. I have not copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part of this submission been written for me by another person or software service.

Signature: Pattarapol Tantechasa

Self-Assessment Details

The following checklists provide an overview of my self-assessment for this unit.

	Pass (P)	Credit (C)	Distinction (D)	High Distinction (Low HD) | (High HD)	
Self-Assessment (please tick)		✓			

Self-assessment Statement

	Included? (tick)
Learning Summary Report	✓
Complete Pass (“core”) task work, approved in Canvas	✓

Minimum Pass Checklist

	Included? (tick)
Additional non-core task work (or equivalent) in a private repository and accessible to staff account.	✓
Spike Extension Report (for spike extensions) in Canvas	✓
Custom Project plan (for D and/or low HD), and/or High HD Research Plan document in Canvas (optional)	

Credit Checklist, in addition to Pass Checklist

	Included? (tick)
Custom Project Distinction Plan document, approved in Canvas	
All associated work (code, data etc.) available to staff (private repository), for non-trivial custom program(s) of own design	
Custom Project “D” level documents in Canvas, to document the program(s) (structure chart etc) including links to repository areas	

Distinction Checklist, in addition to Credit Checklist

	Included? (tick)
Custom Project “HD” level documents in Canvas, to document the program(s) (structure chart etc) including links to repository areas	

Low High Distinction Checklist, in addition to Distinction Checklist

	Included? (tick)
High Distinction Plan document, approved in Canvas	
High Distinction Report document, in Canvas, which includes links to repository assets	
All associated work (code, data etc.) available to staff (private repository) for your research work	

High High Distinction (Research) Checklist, in addition to D/Low HD Checklist

Introduction

This report summarises what I learnt in COS30002 AI for games. It includes a self-assessment against the criteria described in the unit outline, a justification of the pieces included, details of the coverage of the unit intended learning outcomes, and a reflection on my learning.

Overview of Pieces Included

This section outlines the pieces that I have included in my portfolio…

Task 11 - Lab: Steering 1 (Seek, Arrive, Flee)

The foundational piece. The `F=ma` physics integration loop, velocity truncation, and steering force model introduced here run unchanged through every subsequent spike. Included because it is the base every other piece built on. Extensions add Pursuit (predictive targeting) and non-uniform acceleration limits (vehicle-like directional force caps).

Task 12 - Lab: Wander and Path Following

Extends Task 11 with two behaviors directly reused in the spikes: wander (used by the hunter in Task 13 and all agents in Task 14) and sequential waypoint path following (the soldier's patrol route in Task 16). Included to show the progressive build-up of a reusable behavior library.

Task 13 - Spike: Tactical Steering

Hide The prey geometrically evaluates its environment to choose an optimal hiding spot, moving beyond reactive steering into tactical reasoning. Four extensions raise sophistication progressively: obstacle avoidance, multiple hunters, danger-aware path scoring, and a full predator-prey game loop.

Task 14 - Spike: Emergent Group Behavior

Demonstrates how cohesion, separation, alignment, and wander combine via a weighted sum to produce emergent flocking without any scripted group behavior. Four extensions add rigid body constraints, predator avoidance, wall feeler recasting, and a three-species factional ecosystem.

Task 15 - Spike: Agent Marksmanship

Quadratic interception mathematics lets the attacker lead a moving target rather than reacting to its current position. Extensions add a range-closing shotgun attacker, area-of-effect proximity targeting, and a three-state evasion FSM on the target.

Task 16 - Spike: Soldier on Patrol

The most integrated piece in the portfolio. A layered FSM separates decision logic from steering, combining path following, combat, health, and ammo under a formal state architecture. Extensions expand the FSM to four states with hysteresis and three simultaneous concurrent agents.

Task 17 - Spike Extension Report

Consolidates 19 extensions across seven spikes, including assignment spikes (Tasks 4, 6, 10) not individually reproduced here. Included to demonstrate breadth across all five ULOs and to articulate the reasoning behind each architectural decision.

Coverage of the Intended Learning Outcomes

This section outlines how the pieces I have included demonstrate the depth of my understanding in relation to each of the unit's intended learning outcomes.

ILO 1: Software Development for Game AI

"Discuss and implement software development techniques to support the creation of AI behaviour in games"

ILO 1 is about the coding and design decisions that make AI behaviour work cleanly in a game, not just what the AI does, but how the code is structured to support it.

- **Task 11**: Built a single physics loop (`vel += accel * delta`) that every agent across every task reuses. The loop doesn't care what behaviour is running. It just applies whatever force the AI returns. This separation between "how to move" and "where to move" is what made it easy to plug in new behaviours later without rewriting movement code.
- **Task 13**: The World calculates hiding spots each frame and stores them; the prey just reads the result. Keeping the geometry calculation separate from the steering code meant I could change how hiding spots are scored (Extension 3) without touching the prey's movement logic at all.
- **Task 14**: Instead of using if/else to switch between behaviours, all forces are calculated every frame and added together with weights. Adding a new behaviour (like predator fleeing or wall avoidance) only required writing one new method and adding one line to the sum, nothing else needed changing.
- **Task 16**: Used the State design pattern so each state (Patrol, Attack, Flee, Heal) manages its own data and logic. When I needed three soldiers running at the same time, no code changes were needed because each soldier had its own independent state, they didn't share any variables.

ILO 2: Graphs and Path Planning

"Understand and utilise a variety of graph and path planning techniques."

Among the portfolio tasks, ILO 2 is most directly covered by the waypoint path following in Tasks 12 and 16. Deeper graph and pathfinding work was done in the assignment spikes (Tasks 4 and 6), so I am including those here as they are the clearest demonstration of this outcome.

- **Tasks 12 and 16**: Implemented sequential waypoint path following: a `Path` object stores an ordered waypoint list. The agent applies `arrive` toward the current waypoint and advances the index on close approach. This exact code became the soldier's patrol route in Task 16's PatrolState with no modification.
- **Task 4 (Assignment)**: Extended graph search with a visualisation that traces the solution path back to the root by following each node's parent pointer. This shows how graph traversal builds up a path through connected states, not just a single answer.
- **Task 6 (Assignment)**: Applied A* pathfinding to a graph that changes at runtime. When two agents collide, that tile's edge cost spikes and slowly decays. Because A* always picks the lowest-cost path, agents automatically reroute around the hazard which is the same pathfinding algorithm handles both fixed terrain costs and live dynamic threats without any extra logic.

ILO 3: Force-based Agent Movement

"Create realistic movement for agents using steering force models."

- **Task 11**: Implemented Seek, Arrive (three deceleration rates), Flee (panic radius), and Pursuit (predicts future position using `look_ahead = dist / (pursuer_speed + prey_speed)`).
- **Task 12**: Added Wander via a jitter circle projected ahead of the agent. Normalising the drifting target back onto the circle each frame is what makes movement smooth rather than erratic.
- **Task 13**: The prey uses `arrive` (not `seek`) toward its hiding spot to prevent oscillation. Obstacle avoidance blends a repulsion force scaled by penetration depth, plus a hard position correction for full overlaps.
- **Task 14**: Flocking alignment computes the force in velocity-space `(desired_vel - current_vel)` so it scales comparably to seek and flee and can be weight-balanced against them. Wall avoidance used three feeler rays with proximity-scaled steering force.

- **Tasks 15 and 16**: The same seek, flee, and arrive behaviours from Task 11 appear in the shotgun range-closing logic, the FleeState, and the HealingState: reused without modification in far more complex contexts.

ILO 4: Goals and Planning Actions

"Create agents that are capable of planning actions in order to achieve goals."

- **Task 13**: Extension 3 scored each hiding spot by `dist_to_spot + path_exposure × 2.0`, where path exposure accumulates when a hunter is within 150px of the approach route. The prey accepts a detour twice as long for a fully safe route: it reasons about journey risk, not just destination distance.
- **Task 15**: The TargetAgent's three-state FSM (PATROL / EVADE / SEEK_COVER) selects its response based on projectile speed and obstacle proximity. Fast projectiles are deliberately excluded from detection because `flee()` cannot react in time, ignoring them is the correct decision.
- **Task 16**: A four-state FSM with priority Flee > Attack > Heal > Patrol. Hysteresis (enter low-health states at 30hp, exit at 80hp) prevents state thrashing when a soldier takes periodic damage near the boundary: a design decision only discovered after observing rapid oscillation in practice.

ILO 5: Combine AI Techniques

"Combine AI techniques to create more advanced game AI."

- **Task 13**: Combines wander, geometric hide spot projection, arrive steering, danger-aware path scoring, line-of-sight raycasting, and pursuit into a self-sustaining predator-prey loop. No single technique produces this: each layer contributes something the others cannot.
- **Task 14**: Extension 4 produces a three-species food chain (prey, predator, scavenger) using only a faction integer to filter neighbour queries and per-faction steering rules. Complex inter-species dynamics emerge from three simple local rule sets with no global coordination.
- **Task 16**: The final simulation integrates a four-state FSM, waypoint path following, seek/arrive/flee steering, bidirectional projectile combat, a health system, an ammo/reload system, a HealingStation, and three independent concurrent agents. The FSM decides; steering moves; physics handles combat; health and ammo create resource constraints. The result could not arise from any single technique alone.

Reflection

The most important things I leant:

Complex behaviour is composed, not programmed. The most convincing behaviours in this portfolio, a prey choosing a longer but safer escape route, a soldier retreating to heal and returning to fight, a three-species food chain with no inter-species coordination. All emerged from combining simple, independently understandable mechanisms. Emergence does not require complexity in the components; it requires thoughtful composition.

I also learnt not to pre-emptively optimise. Recomputing all hiding spots every frame, testing every agent pair for collision every frame, and raycasting three feelers per agent per frame are all technically unbounded per-frame operations. At 25–60 agents, none were noticeable bottlenecks. Measure before optimising.

The things that helped me most were:

Reusing the codebase from previous tasks. By Task 16 I had working seek, flee, arrive, wander, path following, and projectile combat ready to compose. Later spikes were about integration, not re-implementation.

Real-time debug visualisations (hide spot markers in Task 13, neighbourhood rings in Task 14, predicted interception crosshair in Task 15) caught more bugs faster than logging. Seeing the AI's internal data rendered live made the gap between intended and actual behaviour immediately obvious.

I found the following topics particularly challenging:

The quadratic interception formula in Task 15 was the most mathematically demanding. Understanding why the problem reduces to $at^2 + bt + c = 0$ and why projectile speed appears as a subtraction in coefficient a required working through the geometry on paper first. Initial sign errors produced negative discriminants for all inputs.

State transition design in Task 16 was challenging in a different way: the individual states were straightforward to implement, but designing the correct priority ordering and hysteresis thresholds for four interacting states required careful reasoning. I would not have identified the need for hysteresis without observing state-thrashing in a running simulation first.

I found the following topics particularly interesting:

Emergent collective behaviour in Task 14. The fact that tight schooling, wall navigation, and factional food-chain dynamics all arise from three weighted numbers and a neighbourhood radius, with no agent aware of the group as a collective entity, remains genuinely surprising after having implemented it.

I feel I learnt these topics, concepts, and/or tools really well:

- **The steering force model**: I can implement any steering behaviour from first principles. The desired-minus-current velocity pattern runs unmodified from Task 11 through Task 16.
- **The State FSM pattern**: Task 16's FSM is the cleanest piece of software in the portfolio. Adding Extension 2's two new states required no changes to the agent class or any existing state.
- **Weighted-sum blending:** I understand its trade-offs (emergent and tuneable but can produce conflicting forces) and can tune weights empirically to produce desired emergent modes.

I still need to work on the following areas:

- **Graph-based path planning (ILO 2)**: My path following work uses sequential waypoints, not a graph. The *A** work was completed in assignment spikes and documented in Task 17, but I feel less fluent in those topics than in steering and FSMs. I would benefit from implementing A* from scratch without a provided framework.
- **GOAP**: Task 8 had no extension instructions, so I only completed the baseline simulation. I understand the concept but have limited practical implementation experience compared to FSMs.

My progress in this unit was …:

Consistent. I engaged with each task in sequence, building each spike on top of the previous codebase rather than starting fresh. The clearest evidence of progression is between Tasks 13 and 16: in Task 13 I was still discovering architecture through implementation; by Task 16 I designed the FSM transition table before writing any code.

This unit will help me in the future:

The steering model, FSM architecture, weighted-sum blending, and predictive targeting are standard game AI building blocks I can now implement without reference materials. More broadly, the habit of asking "who should own this state?" before writing a class is a software design practice I will carry into every project.

If I did this unit again, I would do the following things differently:

I would document the graph and A* assignment spikes (Tasks 4 and 6) with the same spike report depth as Tasks 13–16. ILO 2 is the thinnest area of this portfolio, and the most direct way to address it would have been treating those spikes with the same structured design-phase analysis.

I would also add debug visualisations at the start of each spike, not after the logic was written. Several bugs in Tasks 13 and 14 were only found once the visualisations were in place.

Conclusion

In summary, I believe that I have clearly demonstrate that my portfolio is sufficient to be awarded a **Credit** grade.

- **ILO 1**: Demonstrated through principled software design across all spikes: `F=ma` physics, weighted-sum blending, producer-consumer data separation, and the State pattern, each chosen for non-obvious architectural reasons.
- **ILO 2**: Demonstrated through waypoint path following in Tasks 12 and 16, and through graph search tree tracing and dynamic A* cost modification documented in the Task 17 extension report.
- **ILO 3**: Demonstrated through a cumulative steering library built across six tasks, with the same behaviours introduced early reused without modification in later, more complex simulations.
- **ILO 4**: Demonstrated through multi-criteria path scoring in Task 13, FSM-based reactive decision-making in Task 15, and a four-state hysteresis-gated FSM in Task 16.
- **ILO 5**: Demonstrated in the completed extensions of Tasks 13, 14, and 16, each producing emergent behaviour that arises from the deliberate composition of multiple AI techniques.